ETHICAL HACKING LAB

Privacy-safe step-by-step practical guide

Experiments 2, 3, 6 and 7

Based on the uploaded CCL 411 Ethical Hacking Lab manual and the procedures discussed in this session.

Privacy: This edition does not include your name, email address, passwords, authentication tokens, or a specific lab IP address. Replace `<DVWA_TARGET_IP>` with your own lab target IP when performing the steps.

Safety: Use these procedures only against your own local lab applications. Do not expose DVWA or Juice Shop to the public internet. Never include credentials, session cookies, or JWT tokens in reports.

How to use this guide

This guide collects the procedures for the four requested experiments. Record the actual results observed on your system. Any target IP below is represented by a placeholder so the document does not preserve environment-specific details.

Experiments covered

- Experiment 2 — Network Footprinting (Reconnaissance) and Vulnerability Assessment
- Experiment 3 — Test for Configuration and Deployment
- Experiment 6 — Authentication and Authorization Testing
- Experiment 7 — Input Validation Testing (SQL Injection)

Use `<DVWA_TARGET_IP>` in place of the IP address of your own DVWA lab machine.

Experiment 2 — Network Footprinting and Vulnerability Assessment

Aim: Identify the DVWA target IP, perform basic reconnaissance, scan ports and services with Nmap, and assess the web server using Nikto and OWASP ZAP.

A. Confirm the target is reachable

1. Open Kali Linux and launch a terminal.
2. Use `ifconfig` to identify the relevant network interface and target IP.
3. Open Firefox and visit `http://<DVWA_TARGET_IP>/`. Log in if prompted.

B. Passive footprinting

```
whois <DVWA_TARGET_IP>
```

For private IPs, WHOIS may return private-address range information rather than a record identifying the lab host.

```
nslookup <DVWA_TARGET_IP>
```

A reverse lookup may return NXDOMAIN if no reverse-DNS record exists; this can be normal for a private lab IP.

C. Nmap scan

```
nmap -sS -sV -O <DVWA_TARGET_IP>
```

Record host status, open ports, service versions, and any OS-detection result. In the run discussed, the target exposed HTTP on port 80 and reported Apache 2.4.10 (Debian); no exact OS match was detected.

D. Nikto scan

```
nikto -h http://<DVWA_TARGET_IP>/
```

Review the output for server version, directory indexing, configuration exposure, cookie flags, and missing security headers. Treat scanner results as findings to investigate, not all as confirmed exploitable vulnerabilities.

E. OWASP ZAP scan

1. Launch ZAP with `zaproxy`.
2. If ZAP reports that Java is too old, select an installed Java 17+ alternative.
3. In Quick Start, choose Automated Scan.
4. Enter `http://<DVWA_TARGET_IP>/` and click Attack.
5. Open Alerts and review each alert's risk, evidence, and description.

Conclusion: Record the HTTP service, server information, and potential web-application/configuration weaknesses observed in your own scan.

Experiment 3 — Test for Configuration and Deployment

Aim: Use ZAP spidering and passive scanning, inspect robots.txt, and check CVEs actually reported by tools against the NVD. WPScan applies to WordPress sites only.

A. Spider the target in ZAP

1. Open ZAP.
2. Use Quick Start / Manual Explore or the Spider option available in your version.
3. Enter `http://<DVWA_TARGET_IP>/`.
4. Start Spider and wait until progress reaches 100%.
5. Open the Spider tab and review Processed Nodes.

B. Review passive alerts

1. Open Alerts after spidering.
2. Allow passive scanning to process the requests.
3. Select alerts and note the name, risk, evidence, and CWE/WASC references, if shown.

C. Check robots.txt

```
curl http://<DVWA_TARGET_IP>/robots.txt
```

During the session, the file returned `User-agent: *` and `Disallow: /`. This asks crawlers not to crawl the site; robots.txt is not access control.

D. Check CVEs in NVD

1. Visit https://nvd.nist.gov/.
2. Search only for a CVE actually identified by a tool or software version check.
3. Review affected products, impact, and remediation.
4. Do not confuse CWE or WASC IDs with CVE IDs. If no CVE is shown, record that no specific CVE was identified in the examined alert.

E. WPScan (WordPress sites only)

```
wpscan --url http://<target-wordpress-site> --enumerate u,vp,vt
```

Do not run this against DVWA as if it were WordPress. Use only an instructor-provided WordPress lab target.

Conclusion: Record discovered paths, passive-scan alerts, robots.txt output, and any verified CVE references.

Experiment 6 — Authentication and Authorization Testing

Aim: Test login behavior and authorization controls in OWASP Juice Shop using the browser and Burp Suite.

A. Start Juice Shop using the lab manual's Docker command

```
sudo docker run -d -p 3000:3000 bkimminich/juice-shop
```

Open `http://localhost:3000`. If port 3000 is already in use, stop and inspect the existing container instead of creating conflicting instances.

B. Capture the login request in Burp

1. Launch Burp Suite and use its embedded browser, or configure the browser proxy to `127.0.0.1:8080` as described in the manual.
2. Log in to your own test account.
3. Open Proxy → HTTP history.
4. Find `POST /rest/user/login`.
5. Inspect the response without copying tokens or credentials into the report.

C. Compare valid and invalid login

1. Attempt login with valid credentials and record the result.
2. Log out and try an incorrect password for the same test account.
3. Find the second request in Burp and record the response status and message.

Observed during the session: valid login returned a JWT and identified the role as customer; invalid login showed “Invalid email or password.” Do not include the token or account details in a report.

D. Test customer access to administration

1. Log in as the normal customer test account.
2. Visit `http://localhost:3000/#/administration`.
3. Record whether the page is accessible. In the session, it was not accessible through the browser. This does not by itself prove every API endpoint enforces authorization.

E. Check saved-addresses access

1. While logged in, visit `http://localhost:3000/#/saved-addresses`.
2. Log out and revisit the URL.
3. Record whether private data appears. During the session, the route loaded after logout but no private data was shown.

F. Test token removal

1. Log in and open Developer Tools (F12) → Application → Local Storage → `http://localhost:3000`.
2. Delete only the token entry.
3. Refresh and test an account-specific page.
4. Record the final observed behavior. If the page still appears logged in, hard-refresh and recheck the token and application state; do not delete unrelated storage values.

Security note: A JWT is a bearer credential. Never include tokens, passwords, email addresses, or session cookies in screenshots or reports. Refresh the session after a token has been exposed.

Experiment 7 — Input Validation Testing (SQL Injection)

Aim: Test SQL injection in DVWA's SQL Injection module at Low security using the examples listed in the lab manual.

A. Set DVWA security to Low

1. Open `http://<DVWA_TARGET_IP>/` and log in.
2. Click DVWA Security in the left menu.
3. Select Low and click Submit.
4. Click SQL Injection in the left menu.

B. Test a normal ID

```
1
```

During the session, ID 1 returned First name: admin; Surname: admin.

C. Test Boolean-based SQL injection

```
1' OR '1'='1
```

During the session, five records were returned (admin, Gordon Brown, Hack Me, Pablo Picasso, and Bob Smith). This indicates the input changed query logic in the intentionally vulnerable Low-security lab.

D. Retrieve database server version

```
1' UNION SELECT null, version()#
```

During the session, the Surname field showed `5.5.58-0+deb8u1`. This demonstrates retrieval of database version information in the lab.

E. Retrieve the database name

```
1' UNION SELECT null, database()#
```

The shared output showed the original admin row and a second row with a blank Surname field. The database name was not visible, so this test needs verification; do not guess the database name.

F. Remaining manual examples

The manual lists further examples for table names and user/password-related data. Use only the local DVWA lab, and record the exact input and visible output. Never use these tests against real systems without explicit authorization.

Conclusion: The Boolean-based input returned multiple records, and a UNION query exposed the database version. The database-name result remains unconfirmed based on the visible output.

Final checklist for your lab record

Keep screenshots of the command or tool, relevant output, and a short conclusion. Redact all credentials and tokens. The target IP is intentionally represented by a placeholder in this privacy-safe edition.

Experiment	Evidence to keep
Experiment 2	Interface/IP evidence; WHOIS and nslookup outputs; Nmap report; Nikto findings; ZAP Alerts.
Experiment 3	ZAP Spider/Processed Nodes; passive Alerts; robots.txt output; NVD lookup only for CVEs actually reported.
Experiment 6	Burp login result with secrets redacted; valid vs invalid login; administration access; saved-addresses result; verified token-removal result.
Experiment 7	DVWA Low security; normal ID; Boolean injection; version query; database-name query after verification.

Accuracy reminder: This guide reflects the procedures and results discussed. Unconfirmed results are marked as needing verification rather than guessed.